
FlowTrack: Controlling Edit-Signal Execution in Inversion-Free Flow Video Editing

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Text-guided video editing requires applying a semantic change while keeping
2 source motion, layout, and unchanged content stable. Recent inversion-free flow
3 editors derive effective edit signals from target-source velocity differences, yet
4 typically inject these signals directly at each solver step. In neural video sam-
5 pling, however, this direct execution rule overlooks an important distinction: the
6 velocity difference is an online, state-dependent observation rather than a fixed
7 endpoint-displacement command. As the target branch evolves under previous
8 injections, direct injection may accumulate temporal fluctuations and off-target
9 perturbations, producing motion drift, identity flicker, or unintended changes in
10 weak-response regions. We propose FlowTrack, a causal execution controller
11 for inversion-free flow video editing. FlowTrack leaves the pretrained gener-
12 ator and raw edit signal unchanged, but executes the signal through a carried
13 displacement-like direction and response-aware attenuation. We derive this update
14 as the closed-form solution of a local execution objective that balances current
15 signal tracking, temporal carrying, and weak-response suppression. On the full
16 FiVE-Bench protocol, FlowTrack improves motion fidelity, unedited-region fi-
17 delity, and structure consistency over strong training-free diffusion- and flow-based
18 editors while maintaining competitive target-prompt alignment. Code is available
19 at <https://anonymous.4open.science/r/FlowTrack>.

20 1 Introduction

21 Text-guided video editing is difficult because the requested change is usually local in semantics but
22 global in consequences. A successful edit should turn a dog into a plush dog without changing how
23 the dog moves, where the camera goes, or what the beach looks like. Figure 1 shows examples of
24 this source-preserving edit behavior. The target prompt specifies what should change; the source
25 video specifies the layout, motion, camera path, scene content, and temporal correspondences that
26 should remain stable [Qi et al., 2023, Cong et al., 2024, Geyer et al., 2024, Kara et al., 2024, Li
27 et al., 2025b]. In inversion-free editing, this balance must be achieved directly through the sampling
28 trajectory, without reconstructing and optimizing a source-specific latent trajectory.

29 Flow-based generative models offer a promising handle on this problem because they expose velocity
30 fields along the trajectory [Lipman et al., 2023, Liu et al., 2023, Albergo et al., 2025]. Recent flow
31 editors exploit the target-source velocity difference as an edit signal [Kulikov et al., 2025, Li et al.,
32 2025a, Cai et al., 2025]. This signal is not arbitrary: under the linear noise-to-data path used by
33 flow matching, two endpoints sharing the same noise endpoint have ideal velocities whose difference
34 equals the endpoint displacement. In the ideal shared-noise construction, a velocity difference
35 therefore behaves like the displacement from the source endpoint to an edited endpoint.



Figure 1: FlowTrack editing examples. FlowTrack applies target edits while retaining source layout and motion consistency.

36 However, a meaningful edit signal is not yet an executable sampler input. In real neural video editing,
 37 the edited endpoint is not chosen in advance; it is induced online by the prompt-conditioned rollout.
 38 The pretrained flow model provides a state- and time-dependent conditional-mean field, and each
 39 target-source signal has already been affected by previous injected edits. The edit signal moves as
 40 the rollout moves. Treating the whole dense field as equally executable can accumulate temporal
 41 fluctuations along the trajectory and inject weak off-target responses into content that should remain
 42 stable. In videos, these effects can appear as motion drift, identity flicker, or unintended changes in
 43 weak-response regions. Thus a key failure mode of inversion-free flow editing is not necessarily that
 44 the signal is wrong; it is that direct execution lacks a rule for deciding which parts of the online signal
 45 should be carried forward and where they should be attenuated.

46 This motivates an execution view of inversion-free editing. We propose FlowTrack, which keeps
 47 the original velocity-difference signal and the pretrained generator unchanged, but separates signal
 48 estimation from signal execution. Rather than directly injecting every local signal, FlowTrack carries
 49 a persistent displacement-like edit direction across solver steps and uses response-aware execution
 50 support to attenuate regions where the prompt provides weak evidence for change. In short, FlowTrack
 51 contributes a causal execution controller for online velocity-difference signals.

52 We derive FlowTrack as the closed-form minimizer of a local execution objective with three terms:
 53 tracking the current edit signal, maintaining the carried direction, and making injection expensive in
 54 weak-response regions. This framework contains direct execution as the unregularized special case.
 55 On the full FiVE-Bench protocol, the gains are consistent with this execution-control view: FlowTrack
 56 improves motion fidelity, unedited-region fidelity, and structure consistency over strong training-free
 57 diffusion- and flow-based editors, while maintaining competitive target-prompt alignment.

58 Our contributions are:

- 59 • We identify signal execution as an explicit design axis in inversion-free flow video editing.
 60 Velocity differences are geometrically meaningful edit signals, but neural video samplers
 61 observe them online along a closed-loop rollout, so direct execution can amplify drift and
 62 off-target perturbations.
- 63 • We propose FlowTrack, a causal execution controller whose update is the closed-form
 64 optimizer of a local objective that unifies signal tracking, temporal carrying, and weak-
 65 response suppression. This formulation recovers direct execution as an unregularized special
 66 case.

- We validate the execution view on FiVE-Bench. FlowTrack improves motion fidelity, unedited-region fidelity, and structure consistency over strong training-free diffusion and flow baselines, with ablations showing complementary gains from carrying and support.

2 Related Work

Diffusion and flow video generation. Modern video generators are increasingly built on diffusion and flow formulations, where sampling is viewed as trajectory integration under a learned score or velocity field [Sohl-Dickstein et al., 2015, Ho et al., 2020, Song et al., 2021b, Ho and Salimans, 2022, Song et al., 2021a, Dhariwal and Nichol, 2021, Rombach et al., 2022, Peebles and Xie, 2023, Lipman et al., 2023, Liu et al., 2023, Albergo et al., 2025, Tong et al., 2024, Esser et al., 2024, Ho et al., 2022b, Singer et al., 2022, Ho et al., 2022a, WanTeam et al., 2025]. These models provide strong pretrained priors for text-to-video synthesis and editing, but their sampling trajectories are high-dimensional spatiotemporal objects. FlowTrack assumes such a pretrained video model and changes only how an edit signal is executed inside its sampler.

Training-free video editing. Text-guided video editing must apply the requested semantic change while maintaining frame-to-frame correspondence, source layout, object identity, and camera motion. Existing methods address this by reusing pretrained generative priors and adding trajectory-level constraints, attention control, feature propagation, optical-flow alignment, token merging, deformation fields, or noise manipulation [Bar-Tal et al., 2022, Molad et al., 2023, Shin et al., 2024, Wu et al., 2023, Khachatryan et al., 2023, Ceylan et al., 2023, Liu et al., 2024, Qi et al., 2023, Cong et al., 2024, Geyer et al., 2024, Kara et al., 2024, Li et al., 2024, Ouyang et al., 2024]. Recent benchmarks also emphasize evaluation beyond prompt alignment, including motion fidelity, unedited-region stability, and structural deviation [Li et al., 2025b]. FlowTrack follows this video-centric setting: it keeps the pretrained generator and base sampler intact, while controlling the edit-channel input injected into the spatiotemporal latent at each solver step.

Flow-based video editing. Recent flow-based editors exploit target-source velocity differences as inversion-free edit signals. FlowEdit formulates an inversion-free source-to-target ODE for pretrained flow models, and later flow-based video editors steer or modulate this velocity-difference signal for text-to-video editing [Kulikov et al., 2025, Li et al., 2025b,a, Cai et al., 2025]. Related flow-editing analyses also show that inversion-free editing can benefit from trajectory-aware regularization or source-trajectory correction [Wang et al., 2025, Kim et al., 2026, Jiang et al., 2026]. These works primarily focus on constructing, correcting, or steering the edit signal. FlowTrack is complementary: it starts from the same velocity-difference signal, but studies the execution rule that converts the online signal into the sampler input.

Control view of generative editing. Control-theoretic tools provide a language for separating observed signals, state dynamics, and injected controls [Fleming and Rishel, 2012, Bertsekas, 2017]. Generative trajectories can be interpreted as dynamical systems, while inversion-free editing produces velocity-difference signals and response structure online along a rollout affected by the current injected signal. We use this terminology descriptively, to distinguish the learned edit signal from its injection into the sampler. In this local sense, Section 4 defines a causal signal-injection objective whose optimizer exactly matches the FlowTrack update.

Across these lines, prior work improves temporal propagation, signal construction, or signal steering. FlowTrack studies the complementary step of executing a meaningful online edit signal inside the sampler.

3 From Velocity Differences to an Execution Problem

3.1 Why Velocity Differences Are Meaningful Edit Signals

This section separates two questions that are often conflated in inversion-free flow editing: why a velocity difference is a valid edit signal, and how that signal should be executed by a sampler. Linear flow geometry answers the first question through a shared-noise construction. It does not answer the second, because online closed-loop execution does not provide a selected edited endpoint and the pretrained model supplies an online conditional-mean signal rather than a fixed endpoint-displacement command.

Let $x^{\text{source}} \in \mathbb{R}^{F \times C \times H \times W}$ denote the source video latent and let x_k denote the source-anchored edited latent estimate maintained by the inversion-free editing sampler at solver step k . It is not the noise endpoint itself; the time-dependent source and target branches evaluated by the pretrained velocity model are denoted by q_k and p_k . The index k is the flow-solver step, while F is the number of video frames. All Frobenius norms are taken over the full spatiotemporal latent tensor.

Definition 1 (Edit-signal execution variables). At solver step k , let r_k denote the raw velocity-difference signal produced by the pretrained model, m_k denote the carried displacement-like edit direction; initialized as $m_0 = 0$ injected edit-channel input passed to the solver

Symbol	Meaning
r_k	raw velocity-difference signal at solver step k
q_k, p_k	source and target branches evaluated by the velocity model
t_k	reverse sampler time at step k
m_k	carried displacement-like edit direction; initialized as $m_0 = 0$
u_k	injected edit-channel input passed to the solver
a_k	nonnegative diagonal execution-cost metric in the local objective
η_k	pointwise execution coefficient $(1 + \lambda)/(1 + \lambda + a_k)$
γ_k	channel-averaged response magnitude used for support estimation
$\tilde{s}_k$	response-derived execution-support field in $[0, 1]$
t / τ	reverse sampler time / ideal noise-to-data path time

The target-source velocity difference used in inversion-free editing is not an arbitrary signal. Its meaning comes from the shared-noise geometry of the linear path used by flow-matching models. Under the linear flow-matching path

$$x_\tau(y) = (1 - \tau)\varepsilon + \tau y, \quad \tau \in [0, 1], \quad (1)$$

the ideal path velocity is $v_{\text{path}}(y) = y - \varepsilon$. Therefore any two endpoints y^{source} and y^{target} coupled to the same noise endpoint ε satisfy

$$v_{\text{path}}(y^{\text{target}}) - v_{\text{path}}(y^{\text{source}}) = y^{\text{target}} - y^{\text{source}}. \quad (2)$$

This elementary identity explains why velocity differences are valid edit signals under the training path: under shared-noise coupling, a target-source velocity difference has the form of an endpoint displacement. The key condition is not that the realized neural rollout must be globally straight, but that the source and edited endpoints are interpreted under the same noise endpoint.

Under the reverse sampler convention used below, $t = 1 - \tau$, so the same source line is anchored as

$$q_t = (1 - t)x^{\text{source}} + t\varepsilon. \quad (3)$$

An ideal same-noise target branch can be written as $p_t = q_t + (1 - t)d$ for a time-independent displacement d . Thus the implied endpoint displacement

$$\hat{d}_k = \frac{p_k - q_k}{1 - t_k} \quad (4)$$

should be invariant across solver steps in the ideal construction. This diagnostic explains what it would mean for an online branch to remain compatible with the shared-noise geometry, but it is not itself an execution rule. The main point for the method is that a neural velocity difference observed during sampling still needs an injection law before it becomes a sampler input.

3.2 Why Meaningful Signals Are Not Executable Commands

The shared-noise identity explains the signal, not its execution. It does not make the pretrained neural difference an exact edit vector for a particular unknown target video. In text-guided editing, the edited endpoint is not selected in advance; it is induced through the prompt-conditioned sampling trajectory. Let Y denote the clean endpoint random variable and let $X_\tau = (1 - \tau)\varepsilon + \tau Y$ be the corresponding linear flow-matching sample at time τ . Writing the flow time argument explicitly in this subsection, the flow-matching optimum is a conditional-mean field:

$$v^*(x_\tau, \tau, c) = \mathbb{E}[Y - \varepsilon \mid X_\tau = x_\tau, \tau, c]. \quad (5)$$

151 Thus a model difference such as

$$\Delta v_\theta(x_\tau, \tau) = v_\theta(x_\tau, \tau, c_{\text{target}}) - v_\theta(x_\tau, \tau, c_{\text{source}}) \quad (6)$$

152 should be read as a local estimate of how the endpoint distribution should move when the condition
 153 changes from the source prompt to the target prompt. It is meaningful because the shared-noise
 154 identity gives velocity differences an endpoint-displacement interpretation, but it is an online edit-
 155 signal observation rather than an executable displacement command.

156 In the inversion-free sampler, this local edit signal is instantiated by evaluating the same pretrained
 157 model on source-side and target-side branches. The solver time t below follows the reverse sampler
 158 convention, with $t = 1 - \tau$ relative to the noise-to-data variable used for the ideal geometry above.
 159 Using the source branch q_t from Eq. (3), the target-side branch is formed online as

$$p_t = q_t + x_t - x^{\text{source}}, \quad (7)$$

160 where x_t is the current source-anchored edited latent estimate. Let $v_\theta(\cdot, t, c)$ be the pretrained
 161 conditional velocity predictor under condition c . The raw velocity-difference signal is

$$r_t := v_\theta(p_t, t, c_{\text{target}}) - v_\theta(q_t, t, c_{\text{source}}). \quad (8)$$

162 For compact notation, we write the same two-branch difference as

$$\Delta v_\theta(p_k, q_k; c_{\text{source}}, c_{\text{target}}) := v_\theta(p_k, t_k, c_{\text{target}}) - v_\theta(q_k, t_k, c_{\text{source}}). \quad (9)$$

163 Eq. (6) describes the signal conceptually as a target-source velocity difference at a common latent
 164 state. In the actual inversion-free sampler, the two velocities are evaluated on the source and target
 165 branches q_t and p_t , giving Eq. (8). Therefore the discrete signal r_k is not a fixed sequence known
 166 before sampling. It is recomputed online and depends on the current rollout. Since the rollout
 167 has already been affected by earlier injected signals, direct execution feeds a sequence of moving
 168 conditional means back into the sampler. A curved signal sequence is natural; the issue is how to
 169 execute each observed signal as part of a stable edit direction without treating every local fluctuation
 170 as a sampler input. If classifier-free guidance is used, v_θ denotes the guided velocity.

171 3.3 Limits of Direct Execution

172 For a discrete solver with transition operator Ψ_k whose second argument is the edit-channel input,
 173 direct inversion-free execution can be written as

$$x_{k+1} = \Psi_k(x_k, r_k), \quad r_k = \Delta v_\theta(p_k, q_k; c_{\text{source}}, c_{\text{target}}). \quad (10)$$

174 Using Definition 1, direct execution is the unregularized execution policy

$$u_k = r_k, \quad x_{k+1} = \Psi_k(x_k, u_k). \quad (11)$$

175 The learned signal r_k in Eq. (8) is a local prompt-induced velocity difference. Direct execution injects
 176 it into the sampler, but does not enforce step-to-step consistency of the implied endpoint displacement
 177 $\hat{d}_k$ after the solver transition. Hence the executed edit direction can drift even when each r_k is locally
 178 meaningful.

179 This matters in video editing because u_k is injected into the full video latent, not into an isolated
 180 pixel or frame. Once the online field is treated as an executable command at every step, temporal
 181 fluctuations and low-response components are injected together. The former appears as drift of the
 182 carried edit direction; the latter can perturb regions where the prompt-induced response provides
 183 weak evidence for change. These are two sides of the same execution problem. FlowTrack therefore
 184 targets the trajectory-level injection problem: given the online edit signal r_k , choose an injected
 185 signal u_k that follows a persistent edit direction while using the response structure of r_k to calibrate
 186 where injection is supported.

187 4 FlowTrack

188 4.1 Causal Edit-Signal Execution

189 FlowTrack is based on a simple principle: the sampler should follow the edit signal without treating
 190 every local fluctuation as an equally reliable input. As summarized in Figure 2, shared-noise geometry

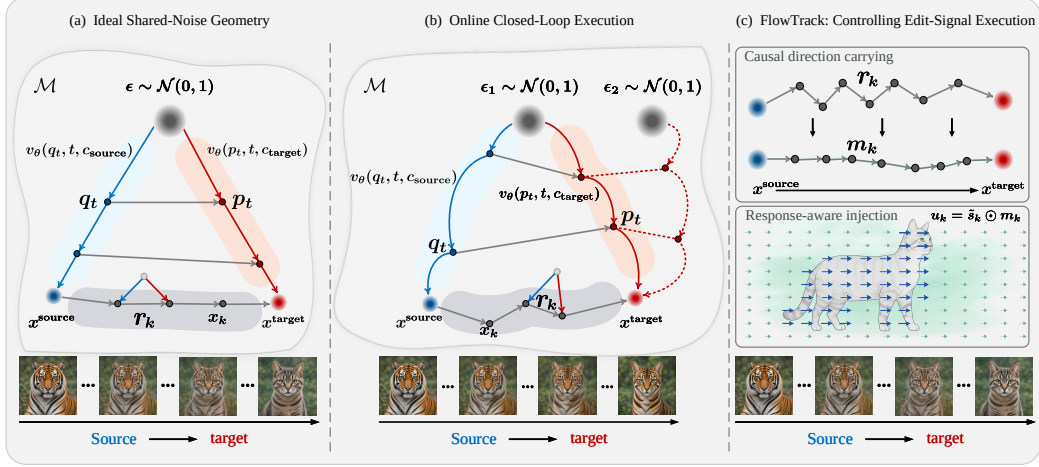


Figure 2: FlowTrack overview. (a) In the ideal shared-noise geometry, a velocity difference behaves like a persistent endpoint displacement. (b) In online closed-loop execution, the velocity-difference signal is re-estimated along a closed-loop trajectory and need not follow one fixed displacement interpretation. (c) FlowTrack keeps the raw signal but executes it through causal direction carrying and response-aware support.

motivates why the velocity-difference signal is meaningful; the missing object is the execution law that maps the online signal r_k into a sampler input u_k . FlowTrack uses the endpoint-displacement structure of the ideal path as a design cue for two local preferences: the injected signal should remain tied to a persistent displacement-like edit direction, and weakly supported components of the dense velocity-difference field should be attenuated.

FlowTrack keeps the raw velocity-difference computation fixed and controls only the injected edit-channel input. The controlled sampler transition is

$$x_{k+1} = \Psi_k(x_k, u_k). \quad (12)$$

FlowTrack is causal: at step k , it computes u_k using only the current signal r_k , the previous carried direction m_{k-1} , and the current response-derived support. It injects the edit signal in two steps. First, it converts the current local signal into a carried displacement-like edit direction

$$m_k := \frac{r_k + \lambda m_{k-1}}{1 + \lambda}. \quad (13)$$

The temporal term keeps injection tied to a persistent source-conditioned edit direction while still anchoring it to r_k . Second, FlowTrack injects this direction through a support-dependent execution coefficient:

$$u_k = \frac{1 + \lambda}{1 + \lambda + a_k} \odot m_k. \quad (14)$$

When a_k is calibrated from a support field $\tilde{s}_k$ in Section 4.4, this becomes $u_k = \tilde{s}_k \odot m_k$. Strong response passes the carried direction more directly, while weak response suppresses off-target perturbations. Thus FlowTrack leaves the raw signal computation unchanged and modifies only the input injected into the sampler.

4.2 Local Execution Objective

The shared-noise geometry gives the signal principle, but not an execution law. During inversion-free neural sampling, the controller observes r_k online along a rollout already affected by previous injections. FlowTrack therefore treats signal injection as a local execution objective over the newly observed evidence r_k , the carried direction m_{k-1} , and a diagonal execution metric a_k derived from the current response structure:

$$\mathcal{L}_k(z) = \frac{1}{2} \|z - r_k\|_F^2 + \frac{\lambda}{2} \|z - m_{k-1}\|_F^2 + \frac{1}{2} \|a_k^{1/2} \odot z\|_F^2. \quad (15)$$

214 This objective encodes the controller’s local preferences. The first term tracks the current evidence
 215 r_k , the second prevents abrupt departure from the carried edit direction m_{k-1} , and the third makes
 216 injection expensive in locations where the prompt provides weak evidence for change. Thus temporal
 217 carrying and conservative support become two terms of the same local execution objective.

218 Direct execution is recovered by removing both control mechanisms, i.e., $\lambda = 0$ and $a_k = 0$, which
 219 gives $u_k = r_k$.

220 4.3 Closed-Form Causal Execution

221 Theorem 1 shows that the online controller in Eqs. (13)–(14) is exactly the pointwise optimizer of
 222 Eq. (15). Its role is structural: direct execution, causal direction filtering, and response-supported
 223 attenuation are special cases or factors of the same execution objective.

224 **Theorem 1** (Closed-form causal execution controller). *For any $\lambda \geq 0$ and any finite nonnegative*
 225 *diagonal execution-cost metric a_k , the local execution objective in Eq. (15) is strictly convex and has*
 226 *the unique optimizer*

$$u_k^* = \frac{r_k + \lambda m_{k-1}}{1 + \lambda + a_k}, \quad (16)$$

227 where division by $1 + \lambda + a_k$ is elementwise.

228 The proof is given in Appendix C.

229 **Consequences and Interpretation.** The closed-form law has three immediate consequences. First,
 230 direct execution is recovered exactly: when $\lambda = 0$ and $a_k = 0$, FlowTrack gives $m_k = u_k = r_k$.
 231 Thus direct execution is the unregularized special case. Second, when $a_k = 0$, the update becomes

$$m_k = u_k = \frac{r_k + \lambda m_{k-1}}{1 + \lambda}, \quad (17)$$

232 which is causal direction carrying. Third, for any finite $a_k \geq 0$, the coefficient in Eq. (14) lies in
 233 $(0, 1]$, so execution-cost calibration can attenuate but cannot amplify the carried direction. Under the
 234 direct support form in Eq. (18), the same no-amplification property holds for all $\tilde{s}_k \in [0, 1]$, including
 235 zero support. Since r_k is produced along a closed-loop rollout and future signals are unavailable at
 236 step k , the memory term carries a displacement-like edit direction, and the support term calibrates the
 237 current injection strength.

238 4.4 Support-Field Calibration

239 We now connect the pointwise execution rule to the response structure of the edit signal. FlowTrack
 240 uses an Otsu response estimator to produce a causal support coefficient $\tilde{s}_k(i) \in [0, 1]$, where larger
 241 values indicate stronger local evidence for applying the carried direction. Given this coefficient,
 242 FlowTrack executes

$$u_k = \tilde{s}_k \odot m_k. \quad (18)$$

243 For $\tilde{s}_k(i) > 0$, this is equivalent to the finite execution-cost metric

$$a_{k,i,c} = (1 + \lambda) \left(\frac{1}{\tilde{s}_k(i)} - 1 \right), \quad (19)$$

244 while $\tilde{s}_k(i) = 0$ is the zero-injection limit. Thus $\tilde{s}_k$ directly acts as the execution coefficient:
 245 high-support locations pass the carried direction, while low-support locations attenuate injection.
 246 The Otsu estimator is annotation-free and is computed from the response magnitude of the current
 247 velocity-difference field; Appendix D gives the smoothing and soft-assignment details.

248 5 Experiments

249 The experiments evaluate whether FlowTrack improves the failure modes discussed above while
 250 preserving target-prompt alignment. We use FiVE-Bench Edit1–6 ($n = 419$) [Li et al., 2025b] and
 251 compare with training-free diffusion- and flow-based editors under the same evaluator and weighted
 252 Overall aggregation. Unless otherwise stated, $\lambda = 0.20$; Appendix E gives the metric definitions,
 253 backbone setting, protocol, and reproduction details.

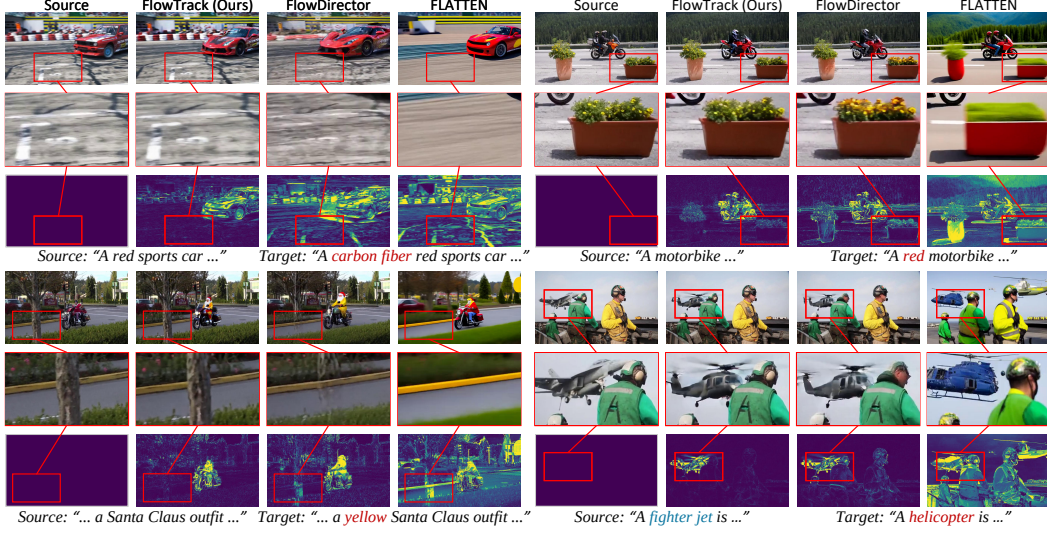


Figure 3: Qualitative comparison. FlowTrack keeps source structure and unedited content stable while applying target changes. Heatmaps show edit-response magnitude; warmer colors indicate stronger response.

5.1 Overall Results

Table 1 reports the Overall split. FlowTrack gives the best MFS, PSNR, LPIPS, MSE, SSIM, and Structure, with a small CLIP trade-off. Figure 3 shows corresponding qualitative behavior. Appendix E.3 reports the Edit1–6 breakdown, and Table 4 gives paired bootstrap intervals.

Table 1: Overall FiVE-Bench results ($n = 419$). Best results are in bold and second-best results are underlined.

Method	MFS $\uparrow$	CLIP $\uparrow$	PSNR $\uparrow$	LPIPS $\downarrow$	MSE $\downarrow$	SSIM $\uparrow$	Structure $\downarrow$
FateZero [Qi et al., 2023]	73.7298	26.0856	18.6122	236.8776	155.9120	69.1964	55.3834
FLATTEN [Cong et al., 2024]	73.9243	26.9928	16.3716	269.9812	255.4401	62.9804	71.3739
FlowDirector [Li et al., 2025a]	85.2422	27.6559	27.4104	<u>52.4975</u>	<u>22.9882</u>	87.7195	8.9845
RAVE [Kara et al., 2024]	82.3514	27.3857	20.8132	137.9647	96.9810	78.1291	25.2783
TokenFlow [Geyer et al., 2024]	81.1081	26.4377	23.0263	137.5256	57.7608	75.4634	26.6105
Wan-Edit [Li et al., 2025b]	83.1296	26.1248	25.5700	71.1971	39.8569	83.4118	12.2960
DFVEdit [Cai et al., 2025]	82.8349	27.1560	25.1480	72.4622	37.9288	82.9375	11.9711
FlowTrack (ours)	86.0860	<u>27.4871</u>	29.2961	39.5408	15.3677	90.2092	5.9808

5.2 Ablations and Sensitivity

Table 2 reports matched structural ablations. Direct execution removes carrying and support; Causal direction only keeps the update in Eq. (17); Support only keeps response-aware attenuation. Table 3 varies λ on a FiVE-Bench subset ($n = 100$). Together, the results show complementary roles: carrying stabilizes the edit direction, support controls where it is injected, and increasing λ improves stability metrics while mildly lowering CLIP. We use $\lambda = 0.20$ by default because it retains stronger target alignment than larger values.

5.3 Statistical Reliability

We estimate uncertainty with a stratified paired bootstrap over matched FiVE-Bench examples. Table 4 reports paired-bootstrap 95% confidence intervals for FlowTrack improvements; lower-is-better metrics are reported as reductions, so positive values always favor FlowTrack. The intervals show reliable gains on motion, unedited-region, and structure metrics, with the expected small CLIP trade-off.

Table 2: Structural ablations on FiVE-Bench ($n = 419$). Best results are in bold and second-best results are underlined.

Method	MFS $\uparrow$	CLIP $\uparrow$	PSNR $\uparrow$	LPIPS $\downarrow$	MSE $\downarrow$	SSIM $\uparrow$	Structure $\downarrow$
Direct execution	83.8333	27.8676	25.0004	75.0427	38.8298	83.0756	12.3624
Causal direction only	83.9296	<u>27.8392</u>	25.1247	73.4815	37.6697	83.3543	12.0138
Support only	<u>85.9782</u>	<u>27.5157</u>	29.1260	40.2292	15.9692	90.1019	<u>6.2062</u>
FlowTrack (ours)	86.0860	27.4871	29.2961	39.5408	15.3677	90.2092	5.9808

Table 3: Sensitivity to the memory strength λ on a FiVE-Bench subset ($n = 100$).

λ	MFS $\uparrow$	CLIP $\uparrow$	PSNR $\uparrow$	LPIPS $\downarrow$	MSE $\downarrow$	SSIM $\uparrow$	Structure $\downarrow$
0	89.0420	27.6360	27.9142	47.8503	2.0565	88.0821	0.6708
0.05	89.0756	27.5780	27.9593	47.5330	2.0394	88.1195	0.6621
0.10	89.1514	27.5512	27.9998	47.2680	2.0172	88.1477	0.6535
0.20	89.1663	27.5472	28.0725	46.9238	1.9882	88.2065	0.6431
0.40	89.1671	27.5205	28.2119	46.3439	1.9343	88.2917	0.6207

Table 4: Paired-bootstrap confidence intervals. Entries are FlowTrack improvement [95% CI]; for all rows, positive is better for FlowTrack.

Main comparisons			
Metric	vs. FlowDirector	vs. Wan-Edit	vs. DFVEdit
MFS gain	+0.8438 [0.6823, 1.0117]	+2.9564 [2.6291, 3.2933]	+3.2511 [2.9071, 3.6145]
CLIP gain	-0.1688 [-0.3185, -0.0233]	+1.3623 [1.1712, 1.5514]	+0.3311 [0.2018, 0.4650]
PSNR gain	+1.8858 [1.7110, 2.0622]	+3.7262 [3.5273, 3.9314]	+4.1482 [4.0032, 4.2907]
LPIPS reduction	+12.9566 [12.1572, 13.7743]	+31.6563 [30.0919, 33.2611]	+32.9213 [31.8727, 34.0214]
MSE reduction	+7.6205 [6.6327, 8.6724]	+24.4892 [22.4367, 26.6831]	+22.5610 [21.1680, 24.0818]
SSIM gain	+2.4897 [2.3306, 2.6523]	+6.7974 [6.4722, 7.1326]	+7.2717 [6.9966, 7.5537]
Structure reduction	+3.0038 [2.6451, 3.3800]	+6.3153 [5.7120, 6.9373]	+5.9904 [5.5757, 6.4219]
Ablations			
Metric	vs. Direct execution	vs. Causal direction only	vs. Support only
MFS gain	+2.2526 [1.9516, 2.5831]	+2.1563 [1.8650, 2.4632]	+0.1078 [0.0289, 0.1914]
CLIP gain	-0.3806 [-0.4655, -0.2947]	-0.3521 [-0.4366, -0.2696]	-0.0286 [-0.0685, 0.0117]
PSNR gain	+4.2958 [4.1714, 4.4209]	+4.1714 [4.0510, 4.2972]	+0.1702 [0.1505, 0.1898]
LPIPS reduction	+35.5019 [34.2960, 36.7019]	+33.9407 [32.8254, 35.0953]	+0.6884 [0.5889, 0.8078]
MSE reduction	+23.4621 [22.0163, 24.9737]	+22.3020 [20.9323, 23.7124]	+0.6014 [0.5242, 0.6823]
SSIM gain	+7.1336 [6.8459, 7.4254]	+6.8548 [6.5776, 7.1352]	+0.1073 [0.0950, 0.1196]
Structure reduction	+6.3817 [6.0393, 6.7235]	+6.0331 [5.7088, 6.3706]	+0.2255 [0.1975, 0.2552]

All intervals exclude zero except CLIP vs. Support only. Approximate bootstrap $p < 0.0001$ except CLIP vs. FlowDirector ($p = 0.0232$), MFS vs. Support only ($p = 0.0060$), and CLIP vs. Support only ($p = 0.1558$).

6 Conclusion and Limitations

FlowTrack shows that a key failure mode of inversion-free flow video editing is not necessarily the edit signal itself, but the rule used to execute it. By turning direct execution into causal, support-aware execution, FlowTrack improves the metrics that video editing most directly stresses: motion fidelity, unedited-region stability, and structure consistency. The method still depends on the pretrained generator and on a causal response proxy for execution support; future work could learn richer support fields or extend execution control to multi-object and long-horizon edits. Because FlowTrack changes only the execution rule, it stays lightweight and compatible with existing pretrained flow editors. More broadly, the results suggest that making execution explicit is a useful axis for improving inversion-free video editing without changing the base model.

References

Michael S. Albergo, Nicholas M. Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. *Journal of Machine Learning Research*, 26(209):1–80, 2025.

284 URL <https://jmlr.org/beta/papers/v26/23-1605.html>.

285 Omer Bar-Tal, Dolev Ofri-Amar, Rafail Fridman, Yoni Kasten, and Tali Dekel. Text2live: Text-driven
286 layered image and video editing. In *European Conference on Computer Vision*, pages 707–723,
287 2022.

288 Dimitri P. Bertsekas. *Dynamic Programming and Optimal Control*, volume 1. Athena Scientific, 4
289 edition, 2017.

290 Lingling Cai, Kang Zhao, Hangjie Yuan, Xiang Wang, Yingya Zhang, and Kejie Huang. Dfvedit:
291 Conditional delta flow vector for zero-shot video editing, 2025. arXiv:2506.20967.

292 Duygu Ceylan, Chun-Hao P. Huang, and Niloy J. Mitra. Pix2video: Video editing using image
293 diffusion. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages
294 23206–23217, 2023.

295 Yuren Cong, Mengmeng Xu, Christian Simon, Shoufa Chen, Jiawei Ren, Yanping Xie, Juan-Manuel
296 Perez-Rua, Bodo Rosenhahn, Tao Xiang, and Sen He. Flatten: Optical flow-guided attention for
297 consistent text-to-video editing. In *International Conference on Learning Representations*, 2024.

298 Prafulla Dhariwal and Alexander Quinn Nichol. Diffusion models beat GANs on image synthesis. In
299 *Advances in Neural Information Processing Systems*, volume 34, pages 8780–8794, 2021.

300 Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam
301 Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, Dustin Podell, Tim Dockhorn, Zion English,
302 and Robin Rombach. Scaling rectified flow transformers for high-resolution image synthesis. In
303 *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings*
304 *of Machine Learning Research*, pages 12606–12633. PMLR, 2024. URL <https://proceedings.mlr.press/v235/esser24a.html>.

306 Wendell H. Fleming and Raymond W. Rishel. *Deterministic and Stochastic Optimal Control*,
307 volume 1. Springer Science & Business Media, 2012.

308 Michal Geyer, Omer Bar-Tal, Shai Bagon, and Tali Dekel. Tokenflow: Consistent diffusion features
309 for consistent video editing. In *International Conference on Learning Representations*, 2024.

310 Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*,
311 2022.

312 Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in*
313 *Neural Information Processing Systems*, volume 33, pages 6840–6851, 2020.

314 Jonathan Ho, William Chan, Chitwan Saharia, Jay Whang, Ruiqi Gao, Alexey Gritsenko, Diederik P.
315 Kingma, Ben Poole, Mohammad Norouzi, David J. Fleet, and Tim Salimans. Imagen video: High
316 definition video generation with diffusion models. *arXiv preprint arXiv:2210.02303*, 2022a.

317 Jonathan Ho, William Chan, Chitwan Saharia, Jay Whang, Ruiqi Gao, Alexey Gritsenko, Diederik P.
318 Kingma, Ben Poole, Mohammad Norouzi, David J. Fleet, and Tim Salimans. Video diffusion
319 models. In *Advances in Neural Information Processing Systems*, volume 35, pages 8633–8646,
320 2022b.

321 Lifan Jiang, Boxi Wu, Yuhang Pei, Tianrun Wu, Yongyuan Chen, Yan Zhao, Shiyu Yu, and Deng Cai.
322 SNR-Edit: Structure-aware noise rectification for inversion-free flow-based editing. *arXiv preprint*
323 *arXiv:2601.19180*, 2026.

324 Ozgur Kara, Bariscan Kurtkaya, Hidir Yesiltepe, James M. Rehg, and Pinar Yanardag. Rave:
325 Randomized noise shuffling for fast and consistent video editing with diffusion models. In
326 *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages
327 6507–6516, 2024.

328 Levon Khachatryan, Andranik Movsisyan, Vahram Tadevosyan, Roberto Henschel, Zhangyang
329 Wang, Shant Navasardyan, and Humphrey Shi. Text2video-zero: Text-to-image diffusion models
330 are zero-shot video generators. In *Proceedings of the IEEE/CVF International Conference on*
331 *Computer Vision*, pages 15954–15964, 2023.

- 332 Jeongsol Kim, Yeobin Hong, Jonghyun Park, and Jong Chul Ye. Flowalign: Trajectory-regularized,
333 inversion-free flow-based image editing. In *International Conference on Learning Representations*,
334 2026. URL <https://openreview.net/forum?id=nytttIJfwW7>.
- 335 Vladimir Kulikov, Matan Kleiner, Inbar Huberman-Spiegelglas, and Tomer Michaeli. Flowedit:
336 Inversion-free text-based editing using pre-trained flow models. In *Proceedings of the IEEE/CVF*
337 *International Conference on Computer Vision*, pages 19721–19730, October 2025.
- 338 Guangzhao Li, Yanming Yang, Chenxi Song, and Chi Zhang. Flowdirector: Training-free flow
339 steering for precise text-to-video editing, 2025a. arXiv:2506.05046.
- 340 Minghan Li, Chenxi Xie, Yichen Wu, Lei Zhang, and Mengyu Wang. Five-bench: A fine-grained
341 video editing benchmark for evaluating emerging diffusion and rectified flow models. In *Proceed-*
342 *ings of the IEEE/CVF International Conference on Computer Vision*, pages 16672–16681, October
343 2025b.
- 344 Xirui Li, Chao Ma, Xiaokang Yang, and Ming-Hsuan Yang. Vidtope: Video token merging for
345 zero-shot video editing. In *Proceedings of the IEEE/CVF Conference on Computer Vision and*
346 *Pattern Recognition*, pages 7486–7495, 2024.
- 347 Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow
348 matching for generative modeling. In *International Conference on Learning Representations*, 2023.
349 URL <https://openreview.net/forum?id=PqvMRDCJT9t>.
- 350 Shaoteng Liu, Yuechen Zhang, Wenbo Li, Zhe Lin, and Jiaya Jia. Video-p2p: Video editing with
351 cross-attention control. In *Proceedings of the IEEE/CVF Conference on Computer Vision and*
352 *Pattern Recognition*, pages 8599–8608, 2024.
- 353 Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and
354 transfer data with rectified flow. In *International Conference on Learning Representations*, 2023.
355 URL <https://openreview.net/forum?id=XVjTT1nw5z>.
- 356 Eyal Molad, Eliahu Horwitz, Dani Valevski, Alex Rav-Acha, Yossi Matias, Yael Pritch, Yaniv
357 Leviathan, and Yedid Hoshen. Dreamix: Video diffusion models are general video editors. *arXiv*
358 *preprint arXiv:2302.01329*, 2023.
- 359 Nobuyuki Otsu. A threshold selection method from gray-level histograms. *IEEE Transactions on*
360 *Systems, Man, and Cybernetics*, 9(1):62–66, 1979.
- 361 Hao Ouyang, Qiuyu Wang, Yuxi Xiao, Qingyan Bai, Juntao Zhang, Kecheng Zheng, Xiaowei Zhou,
362 Qifeng Chen, and Yujun Shen. CoDeF: Content deformation fields for temporally consistent
363 video processing. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern*
364 *Recognition*, pages 8089–8099, 2024.
- 365 William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of*
366 *the IEEE/CVF International Conference on Computer Vision*, pages 4195–4205, 2023.
- 367 Chenyang Qi, Xiaodong Cun, Yong Zhang, Chenyang Lei, Xintao Wang, Ying Shan, and Qifeng
368 Chen. Fatezero: Fusing attentions for zero-shot text-based video editing. In *Proceedings of the*
369 *IEEE/CVF International Conference on Computer Vision*, pages 15932–15942, 2023.
- 370 Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Bjorn Ommer. High-
371 resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Confer-*
372 *ence on Computer Vision and Pattern Recognition*, pages 10684–10695, 2022.
- 373 Chaehun Shin, Heeseung Kim, Che Hyun Lee, Sang-gil Lee, and Sungroh Yoon. Edit-a-video: Single
374 video editing with object-aware consistency. In *Proceedings of the 15th Asian Conference on*
375 *Machine Learning*, volume 222 of *Proceedings of Machine Learning Research*, pages 1215–1230.
376 PMLR, 2024. URL <https://proceedings.mlr.press/v222/shin24a.html>.
- 377 Uriel Singer, Adam Polyak, Thomas Hayes, Xiaoyue Yin, Jie An, Songyang Zhang, Qiyan Hu,
378 Harry Yang, Oron Ashual, Oran Gafni, et al. Make-a-video: Text-to-video generation without
379 text-video data. *arXiv preprint arXiv:2209.14792*, 2022.

- 380 Jascha Sohl-Dickstein, Eric A. Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised
381 learning using nonequilibrium thermodynamics. In *International Conference on Machine Learning*,
382 pages 2256–2265, 2015.
- 383 Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *Inter-
384 national Conference on Learning Representations*, 2021a. URL [https://openreview.net/
385 forum?id=St1giarCHLP](https://openreview.net/forum?id=St1giarCHLP).
- 386 Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben
387 Poole. Score-based generative modeling through stochastic differential equations. In *International
388 Conference on Learning Representations*, 2021b. URL [https://openreview.net/forum?id=
389 PXTIG12RRHS](https://openreview.net/forum?id=PXTIG12RRHS).
- 390 Alexander Tong, Nikolay Malkin, Guillaume Huguette, Yanlei Zhang, Jarrid Rector-Brooks, Kilian
391 Fatras, Guy Wolf, and Yoshua Bengio. Improving and generalizing flow-based generative models
392 with minibatch optimal transport. *Transactions on Machine Learning Research*, 2024.
- 393 Jiangshan Wang, Junfu Pu, Zhongang Qi, Jiayi Guo, Yue Ma, Nisha Huang, Yuxin Chen, Xiu Li,
394 and Ying Shan. Taming rectified flow for inversion and editing. In *Proceedings of the 42nd
395 International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning
396 Research*, pages 64044–64058. PMLR, 2025. URL [https://proceedings.mlr.press/v267/
397 wang25ce.html](https://proceedings.mlr.press/v267/wang25ce.html).
- 398 WanTeam, Ang Wang, Baole Ai, Bin Wen, Chaojie Mao, Chen-Wei Xie, et al. Wan: Open and
399 advanced large-scale video generative models, 2025. arXiv:2503.20314.
- 400 Jay Zhangjie Wu, Yixiao Ge, Xintao Wang, Stan Weixian Lei, Yuchao Gu, Yufei Shi, Wynne Hsu,
401 Ying Shan, Xiaohu Qie, and Mike Zheng Shou. Tune-a-video: One-shot tuning of image diffusion
402 models for text-to-video generation. In *Proceedings of the IEEE/CVF International Conference on
403 Computer Vision*, pages 7623–7633, 2023.

404 Appendix

405 A Shared-Noise Geometry and Diagnostic Equivalences

406 **Lemma A.1** (Endpoint-displacement identity). *Consider the linear flow-matching path*

$$x_\tau(y) = (1 - \tau)\varepsilon + \tau y, \quad v_{\text{path}}(y) = \frac{dx_\tau(y)}{d\tau} = y - \varepsilon.$$

407 *For any two endpoints y^{source} and y^{target} coupled to the same noise endpoint ε ,*

$$v_{\text{path}}(y^{\text{target}}) - v_{\text{path}}(y^{\text{source}}) = y^{\text{target}} - y^{\text{source}}.$$

408 *Proof.* The identity follows by subtraction:

$$(y^{\text{target}} - \varepsilon) - (y^{\text{source}} - \varepsilon) = y^{\text{target}} - y^{\text{source}}.$$

409 Thus the velocity difference along the ideal path equals the endpoint displacement under the same-
410 noise coupling. $\square$

411 **Lemma A.2** (Equivalent same-noise displacement diagnostics). *Fix a source branch q_k at solver*
412 *times $t_k < 1$ and a target branch p_k . Define*

$$\hat{d}_k = \frac{p_k - q_k}{1 - t_k}.$$

413 *The following statements are equivalent:*

414 1. *There exists a time-independent displacement d such that*

$$p_k = q_k + (1 - t_k)d, \quad \forall k.$$

415 2. *The implied endpoint displacement $\hat{d}_k$ is invariant across solver steps.*

416 3. *For every adjacent pair of solver steps,*

$$p_{k+1} = q_{k+1} + (1 - t_{k+1})\hat{d}_k.$$

417 *Proof.* If the first statement holds, then

$$\hat{d}_k = \frac{q_k + (1 - t_k)d - q_k}{1 - t_k} = d$$

418 for all k , proving invariance. If $\hat{d}_k$ is invariant, let its common value be d ; then $p_k = q_k + (1 - t_k)d$
419 for all k . The affine-continuation condition is exactly the adjacent-step form of the same equality
420 with $d = \hat{d}_k$. Since it implies $\hat{d}_{k+1} = \hat{d}_k$ for every adjacent pair, it is equivalent to invariance over
421 the discrete trajectory. $\square$

422 **Lemma A.3** (Inverse-noise equivalence). *Assume the final edited endpoint is $z^{\text{target}} = p_T$ with*
423 *$t_T = 0$, and the source branch is $q_k = (1 - t_k)x^{\text{source}} + t_k\varepsilon$. For all intermediate steps with $t_k > 0$,*
424 *define*

$$\hat{\varepsilon}_k^{\text{target}} = \frac{p_k - (1 - t_k)z^{\text{target}}}{t_k}.$$

425 *Then the fixed-displacement diagnostic in Lemma A.2 is equivalent to $\hat{\varepsilon}_k^{\text{target}} = \varepsilon$ for all intermediate*
426 *steps.*

427 *Proof.* If $p_k = q_k + (1 - t_k)d$ and $p_T = z^{\text{target}}$ with $t_T = 0$, then $d = z^{\text{target}} - x^{\text{source}}$. Hence

$$p_k = (1 - t_k)x^{\text{source}} + t_k\varepsilon + (1 - t_k)(z^{\text{target}} - x^{\text{source}}) = (1 - t_k)z^{\text{target}} + t_k\varepsilon,$$

428 which gives $\hat{\varepsilon}_k^{\text{target}} = \varepsilon$. Conversely, if $\hat{\varepsilon}_k^{\text{target}} = \varepsilon$, then

$$p_k = (1 - t_k)z^{\text{target}} + t_k\varepsilon = q_k + (1 - t_k)(z^{\text{target}} - x^{\text{source}}),$$

429 which is a fixed-displacement representation. $\square$

430 **Proposition A.1** (Unconstrained diagnostic mismatch under direct execution). *Let the implied-*
431 *displacement mismatch after step k be*

$$\rho_{k+1} = p_{k+1} - q_{k+1} - (1 - t_{k+1})\hat{d}_k,$$

432 *or equivalently the normalized turn mismatch*

$$e_{k+1} = \frac{p_{k+1} - q_{k+1}}{1 - t_{k+1}} - \frac{p_k - q_k}{1 - t_k}.$$

433 *The direct-execution input law $u_k = r_k$ imposes no constraint or penalty that forces $\rho_{k+1} = 0$ or*
434 *$e_{k+1} = 0$.*

435 *Proof.* Direct execution specifies only the edit-channel input $u_k = r_k$ and advances the sampler
436 as $x_{k+1} = \Psi_k(x_k, u_k)$. The policy does not compute $\hat{d}_k$, does not project p_{k+1} onto the affine
437 continuation in Lemma A.2, and does not optimize an objective containing ρ_{k+1} or e_{k+1} . Therefore
438 these diagnostic mismatches are unconstrained by the execution law itself. This proposition does
439 not claim that the mismatches must be nonzero; it states that direct execution does not enforce their
440 vanishing. $\square$

441 B Online Conditional-Mean Interpretation

442 **Lemma B.1** (Flow-matching optimum as a conditional mean). *Suppose a flow model is trained with*
443 *the squared flow-matching objective*

$$\min_v \mathbb{E}[\|v(X_\tau, \tau, c) - (Y - \varepsilon)\|_F^2].$$

444 *For each (x_τ, τ, c) , the pointwise minimizer is*

$$v^*(x_\tau, \tau, c) = \mathbb{E}[Y - \varepsilon \mid X_\tau = x_\tau, \tau, c].$$

445 *Proof.* Condition on $(X_\tau, \tau, c) = (x_\tau, \tau, c)$ and write $Z = Y - \varepsilon$. The conditional risk is

$$\mathbb{E}[\|z - Z\|_F^2 \mid X_\tau = x_\tau, \tau, c],$$

446 viewed as a function of z . The unique minimizer of squared loss is the conditional expectation, so

$$z^* = \mathbb{E}[Z \mid X_\tau = x_\tau, \tau, c].$$

447 Substituting $Z = Y - \varepsilon$ gives the stated optimum. $\square$

448 **Corollary B.1** (Model differences are distributional signals). *At a common latent state, the ideal*
449 *target-source model difference is*

$$\Delta v^*(x_\tau, \tau) = \mathbb{E}[Y - \varepsilon \mid X_\tau = x_\tau, \tau, c_{\text{target}}] - \mathbb{E}[Y - \varepsilon \mid X_\tau = x_\tau, \tau, c_{\text{source}}].$$

450 *It compares conditional endpoint means under two prompts and need not equal the displacement to a*
451 *single selected target endpoint.*

452 C Controller Properties and Causal Execution

453 **Proposition C.1** (Strict convexity and closed-form optimizer). *For any $\lambda \geq 0$ and finite nonnegative*
454 *diagonal metric a_k , the local objective in Eq. (15) is strictly convex and has the unique optimizer in*
455 *Eq. (16).*

456 *Proof.* The gradient of Eq. (15) is

$$\nabla_z \mathcal{L}_k(z) = z - r_k + \lambda(z - m_{k-1}) + a_k \odot z.$$

457 The Hessian is diagonal with entries $1 + \lambda + a_k(i) > 0$, so the objective is strictly convex. Setting
458 the gradient to zero gives

$$(1 + \lambda + a_k) \odot z = r_k + \lambda m_{k-1}.$$

459 Therefore the unique optimizer is

$$u_k^* = \frac{r_k + \lambda m_{k-1}}{1 + \lambda + a_k},$$

460 where division is elementwise. This proves Theorem 1. $\square$

461 **Corollary C.1** (Carrying and execution factorization). *The optimizer factors into a causal carried-*
 462 *direction update followed by a pointwise execution coefficient:*

$$m_k = \frac{r_k + \lambda m_{k-1}}{1 + \lambda}, \quad u_k = \frac{1 + \lambda}{1 + \lambda + a_k} \odot m_k.$$

463 *Proof.* Substitute the definition of m_k into the optimizer in Proposition C.1:

$$\frac{r_k + \lambda m_{k-1}}{1 + \lambda + a_k} = \frac{1 + \lambda}{1 + \lambda + a_k} \odot \frac{r_k + \lambda m_{k-1}}{1 + \lambda}.$$

464 □

465 **Corollary C.2** (Controller reductions). *The local objective recovers the structural ablations used in*
 466 *Table 2:*

$$\begin{aligned} \lambda = 0, a_k = 0 &\Rightarrow u_k = r_k && \text{(direct execution),} \\ a_k = 0 &\Rightarrow u_k = m_k = \frac{r_k + \lambda m_{k-1}}{1 + \lambda} && \text{(causal direction only),} \\ \text{temporal carrying disabled} &\Rightarrow u_k = \frac{1 + \lambda}{1 + \lambda + a_k} \odot r_k && \text{(support only).} \end{aligned}$$

467 *Under the calibration in Eq. (19), the support-only case becomes $u_k = \tilde{s}_k \odot r_k$.*

468 **Corollary C.3** (Exponential memory form). *Let*

$$\alpha = \frac{1}{1 + \lambda}, \quad \beta = \frac{\lambda}{1 + \lambda}.$$

469 *Then the carried direction obeys*

$$m_k = \alpha r_k + \beta m_{k-1} = \alpha \sum_{j=1}^k \beta^{k-j} r_j + \beta^k m_0.$$

470 *If $m_0 = 0$, then*

$$m_k = \alpha \sum_{j=1}^k \beta^{k-j} r_j.$$

471 *Proof.* The one-step update follows directly from Eq. (13). Repeated substitution gives the finite
 472 expansion. □

473 **Corollary C.4** (Executed-input memory form). *Let*

$$\eta_k = \frac{1 + \lambda}{1 + \lambda + a_k}$$

474 *denote the pointwise execution coefficient in Eq. (14). Then the injected input can be written as*

$$u_k = \eta_k \odot \left(\alpha \sum_{j=1}^k \beta^{k-j} r_j + \beta^k m_0 \right),$$

475 *with α and β as defined in Corollary C.3. If $m_0 = 0$, then*

$$u_k = \eta_k \odot \left(\alpha \sum_{j=1}^k \beta^{k-j} r_j \right).$$

476 *Proof.* By Corollary C.1, $u_k = \eta_k \odot m_k$. Substituting the expansion of m_k from Corollary C.3 gives
 477 the result. □

478 **Corollary C.5** (Turn contraction). *The carried-direction update contracts the instantaneous turn*
 479 *from m_{k-1} toward r_k :*

$$\|m_k - m_{k-1}\|_F = \frac{1}{1 + \lambda} \|r_k - m_{k-1}\|_F.$$

480 *When $a_k = 0$, this is also the contraction of the injected input $u_k = m_k$.*

481 *Proof.* Subtract m_{k-1} from Eq. (13):

$$m_k - m_{k-1} = \frac{r_k - m_{k-1}}{1 + \lambda}.$$

482 Taking Frobenius norms gives the result. $\square$

483 **Proposition C.2** (Decomposition of injected-input variation). *Let $u_k = \eta_k \odot m_k$ with η_k defined in*
 484 *Corollary C.4. Then for every $k \geq 2$,*

$$\|u_k - u_{k-1}\|_F \leq \|\eta_k \odot (m_k - m_{k-1})\|_F + \|(\eta_k - \eta_{k-1}) \odot m_{k-1}\|_F.$$

485 *Moreover,*

$$\|\eta_k \odot (m_k - m_{k-1})\|_F \leq \frac{1}{1 + \lambda} \|r_k - m_{k-1}\|_F.$$

486 *Proof.* Add and subtract $\eta_k \odot m_{k-1}$:

$$u_k - u_{k-1} = \eta_k \odot (m_k - m_{k-1}) + (\eta_k - \eta_{k-1}) \odot m_{k-1}.$$

487 The first inequality follows from the triangle inequality. Since η_k is pointwise in $(0, 1]$, the first term
 488 satisfies

$$\|\eta_k \odot (m_k - m_{k-1})\|_F \leq \|m_k - m_{k-1}\|_F.$$

489 Applying Corollary C.5 completes the proof. $\square$

490 **Proposition C.3** (Bounded-input property). *Assume $\|r_k\|_F \leq R$ for all k and $\|m_0\|_F \leq R$. Then*
 491 *$\|m_k\|_F \leq R$ for all k . If additionally $a_k \geq 0$, then $\|u_k\|_F \leq R$.*

492 *Proof.* The carried update is $m_k = \alpha r_k + \beta m_{k-1}$ with $\alpha, \beta \geq 0$ and $\alpha + \beta = 1$. If $\|m_{k-1}\|_F \leq R$,
 493 then

$$\|m_k\|_F \leq \alpha \|r_k\|_F + \beta \|m_{k-1}\|_F \leq R.$$

494 Induction proves the bound on m_k . Since $a_k \geq 0$, the execution coefficient

$$\eta_k = \frac{1 + \lambda}{1 + \lambda + a_k}$$

495 is pointwise in $(0, 1]$, hence $\|u_k\|_F = \|\eta_k \odot m_k\|_F \leq \|m_k\|_F \leq R$. $\square$

496 **Proposition C.4** (Proximal and MAP interpretation). *The local objective in Eq. (15) is equivalent,*
 497 *up to constants independent of z , to a Gaussian posterior negative log-density in which r_k is a unit-*
 498 *variance observation of the executable input, m_{k-1} is a temporal prior with precision λ , and a_k is a*
 499 *zero-centered execution-cost precision. Equivalently, FlowTrack computes a proximal least-squares*
 500 *estimate that balances the current observation, temporal carrying, and conservative injection.*

501 *Proof.* Consider the unnormalized log density

$$\log p(z \mid r_k, m_{k-1}, a_k) = -\frac{1}{2} \|z - r_k\|_F^2 - \frac{\lambda}{2} \|z - m_{k-1}\|_F^2 - \frac{1}{2} \|a_k^{1/2} \odot z\|_F^2 + \text{const},$$

502 where a_k is a nonnegative diagonal precision. Maximizing this density is identical to minimizing
 503 Eq. (15). Proposition C.1 gives the unique minimizer, hence the FlowTrack update is the correspond-
 504 ing MAP or proximal estimate. $\square$

505 **Definition C.1** (Online filtration and causal policy). Let

$$\mathcal{F}_k = \sigma(x_1, \dots, x_k, r_1, \dots, r_k, \tilde{s}_1, \dots, \tilde{s}_k, m_0)$$

506 be the information available up to solver step k . A policy is causal if u_k is $\mathcal{F}_k$ -measurable.

507 **Proposition C.5** (FlowTrack is causal). *FlowTrack defines a causal policy under Definition C.1.*

508 *Proof.* By induction, m_{k-1} is $\mathcal{F}_{k-1}$ -measurable and therefore $\mathcal{F}_k$ -measurable. The update $m_k =$
 509 $f(r_k, m_{k-1})$ uses only $\mathcal{F}_k$ -measurable quantities, and the injected input $u_k = g(m_k, \tilde{s}_k)$ also uses
 510 only quantities in $\mathcal{F}_k$. Hence u_k is $\mathcal{F}_k$ -measurable. $\square$

D Support-Field Calibration and Estimator

Proposition D.1 (Support-cost equivalence). *FlowTrack uses the direct support execution $u_k = \tilde{s}_k \odot m_k$ with $\tilde{s}_k(i) \in [0, 1]$. For any positive support coefficient $\tilde{s}_k(i) \in (0, 1]$, the channel-broadcast metric*

$$a_{k,i,c} = (1 + \lambda) \left(\frac{1}{\tilde{s}_k(i)} - 1 \right)$$

is finite and nonnegative, and the execution coefficient satisfies

$$\frac{1 + \lambda}{1 + \lambda + a_{k,i,c}} = \tilde{s}_k(i).$$

Conversely, any finite $a_{k,i,c} \geq 0$ induces the positive coefficient

$$\eta_{k,i,c} = \frac{1 + \lambda}{1 + \lambda + a_{k,i,c}} \in (0, 1].$$

The zero-support case $\tilde{s}_k(i) = 0$ is handled directly by $u_k(i, c) = 0$, equivalently as the limit $a_{k,i,c} \rightarrow \infty$.

Proof. Since $\tilde{s}_k(i) \leq 1$, we have $1/\tilde{s}_k(i) - 1 \geq 0$, so $a_{k,i,c} \geq 0$. Since $\tilde{s}_k(i) > 0$, the metric is finite. Substitution gives

$$\frac{1 + \lambda}{1 + \lambda + (1 + \lambda)(1/\tilde{s}_k(i) - 1)} = \frac{1 + \lambda}{(1 + \lambda)/\tilde{s}_k(i)} = \tilde{s}_k(i).$$

The converse follows immediately from $a_{k,i,c} \geq 0$. Taking $\tilde{s}_k(i) \downarrow 0$ makes $a_{k,i,c} \rightarrow \infty$ and the execution coefficient tend to zero. $\square$

Corollary D.1 (Monotonicity). *If $\tilde{s}_1(i) \leq \tilde{s}_2(i)$, then the calibrated costs satisfy $a_1(i) \geq a_2(i)$.*

Proof. The map $s \mapsto (1 + \lambda)(1/s - 1)$ is decreasing on $s > 0$. $\square$

Corollary D.2 (Conservative support execution). *Calibrated support execution can attenuate but cannot amplify the carried direction:*

$$|u_k(i, c)| \leq |m_k(i, c)| \quad \text{for all } i, c.$$

Proof. Under Eq. (18), $u_k(i, c) = \tilde{s}_k(i)m_k(i, c)$ with $\tilde{s}_k(i) \in [0, 1]$. $\square$

Corollary D.3 (Monotone injected energy under support). *Fix the carried direction m_k . If two support fields satisfy $\tilde{s}_k^{(1)}(i) \leq \tilde{s}_k^{(2)}(i)$ for every location i , then their calibrated injected inputs satisfy*

$$\|u_k^{(1)}\|_F \leq \|u_k^{(2)}\|_F \leq \|m_k\|_F.$$

In particular, $\tilde{s}_k \equiv 1$ recovers $u_k = m_k$, while $\tilde{s}_k \equiv 0$ gives zero injection.

Proof. Under calibrated support execution, $u_k^{(\ell)} = \tilde{s}_k^{(\ell)} \odot m_k$ for $\ell \in \{1, 2\}$, with the spatial support broadcast over channels. Since $0 \leq \tilde{s}_k^{(1)}(i) \leq \tilde{s}_k^{(2)}(i) \leq 1$, every squared component satisfies

$$(\tilde{s}_k^{(1)}(i)m_k(i, c))^2 \leq (\tilde{s}_k^{(2)}(i)m_k(i, c))^2 \leq m_k(i, c)^2.$$

Summing over all positions and channels and taking square roots gives the norm inequality. The two limiting cases follow by substituting $\tilde{s}_k \equiv 1$ and $\tilde{s}_k \equiv 0$. $\square$

535 D.1 Otsu Response-Support Estimator

536 We estimate response support from the current raw edit signal. Let i index all non-channel latent
537 positions. Define a channel-shared response magnitude

$$\gamma_k(i) = \frac{1}{C} \sum_{c=1}^C |(r_k)_{i,c}|. \quad (\text{D.1})$$

538 We apply a light local average to this response map and compute an Otsu threshold θ_k from the
539 resulting one-dimensional statistic over latent locations [Otsu, 1979]. If the response statistic is
540 degenerate and no meaningful split exists, we set $\tilde{s}_k(i) = 1$ for all i , which makes the execution-cost
541 metric inactive. Otherwise, we convert the thresholded response into a soft support coefficient,

$$\tilde{s}_k(i) = \sigma \left(\kappa \frac{\gamma_k(i) - \theta_k}{\theta_k + \epsilon} \right), \quad (\text{D.2})$$

542 where κ controls assignment sharpness and ϵ is a small numerical constant. High-response locations
543 receive larger support and low-response locations receive smaller support, yielding $\tilde{s}_k \in [0, 1]$ in
544 Eq. (18).

545 E Algorithm and Experimental Details

Algorithm 1: FlowTrack Execution Controller

Input: source latent x^{source} , time schedule $\{t_k\}_{k=1}^K$, velocity-difference signal Δv_θ , solver transition Ψ_k , memory strength λ , support mode.

Initialize: $x_1 \leftarrow x^{\text{source}}$ and $m_0 \leftarrow 0$.

For $k = 1, \dots, K$:

1. Form the time-dependent source and target branches q_k, p_k from the source-anchored edited estimate x_k using Eq. (7).

2. Compute $r_k \leftarrow \Delta v_\theta(p_k, q_k; c_{\text{source}}, c_{\text{target}})$.

3. Set $m_k \leftarrow (r_k + \lambda m_{k-1}) / (1 + \lambda)$.

4. If support is enabled, infer $\tilde{s}_k$ from the current response and set $u_k \leftarrow \tilde{s}_k \odot m_k$; otherwise set $u_k \leftarrow m_k$.

5. Advance the sampler with $x_{k+1} \leftarrow \Psi_k(x_k, u_k)$.

Return: edited latent estimate x_{K+1} .

546 **Ablation Mapping.** The structural ablations remove terms from the same local objective rather
547 than toggling unrelated heuristics. Direct execution removes temporal carrying and disables support,
548 giving $u_k = r_k$. Causal direction only enables temporal carrying and disables support. Support only
549 disables temporal carrying and keeps calibrated support active, giving $u_k = \tilde{s}_k \odot r_k$. Full FlowTrack
550 enables both temporal carrying and calibrated support.

Table E.1: Evaluation protocol used for the overall benchmark and FlowTrack runs.

Item	Setting
Backbone	Wan2.1-T2V-1.3B for FlowTrack and all flow-based methods
Benchmark	FiVE-Bench, weighted Overall, $n = 419$
FlowTrack setting	Default execution controller, $\lambda = 0.20$
Primary ablations	Structural controller reductions under a matched generation protocol

551 **Metrics.** FiVE-Bench reports editing quality from complementary perspectives. MFS measures
552 source-motion fidelity, and CLIP measures target-prompt alignment. PSNR, LPIPS, MSE, and SSIM
553 are computed on unedited regions to evaluate preservation of unchanged content. Structure distance
554 measures global structural deviation. We follow the FiVE-Bench convention: higher is better for
555 MFS, CLIP, PSNR, and SSIM, while lower is better for LPIPS, MSE, and Structure.

556 **Compute Resources.** FlowTrack is training-free and does not require model finetuning or additional
 557 parameter learning. The reported generation and evaluation jobs were run on a machine with eight
 558 NVIDIA L20 GPUs, each with 46GB memory. A representative 100-video edit-type shard used about
 559 17GB peak GPU memory and about 14 hours of wall-clock time. Since edit types and examples are
 560 independent, full benchmark and ablation sweeps can be parallelized across available GPUs. The
 561 supplemental material provides reproduction instructions and evaluation details.

Table E.2: Compute resources for reproducing FlowTrack experiments.

Item	Setting
Hardware	NVIDIA L20 GPU, 46GB memory
Software	CUDA 12.4, NVIDIA driver 550.67
Training	None; FlowTrack is training-free
Peak GPU memory	About 17GB for a representative run
Representative shard runtime	About 14 hours for a 100-video edit-type shard
Parallelism	Independent over edit types and examples

562 E.1 Execution-Drift Diagnostics

563 To isolate the execution mechanism from final video quality, we additionally report a controlled
 564 latent-trajectory diagnostic. All variants use the same raw online edit signal and differ only in the
 565 execution rule mapping r_k to u_k . The diagnostic is not a FiVE-Bench video-quality score; it measures
 566 whether the executed latent trajectory follows the mechanism predicted by the local objective.

567 For each solver step, we measure the implied-displacement turn

$$e_{k+1} = \frac{p_{k+1} - q_{k+1}}{1 - t_{k+1}} - \frac{p_k - q_k}{1 - t_k}, \quad (\text{E.1})$$

568 and the injected-direction variation

$$\delta_k = 1 - \frac{\langle u_k, u_{k-1} \rangle}{\|u_k\|_F \|u_{k-1}\|_F}. \quad (\text{E.2})$$

569 The first quantity tests whether the executed branch behaves like one carried endpoint displacement;
 570 the second tests whether the injected edit direction flickers across steps. We use standard numerical
 571 stabilization for zero-energy steps in implementation. We also measure response leakage as

$$\ell_k = \frac{\|(1 - h_k) \odot u_k\|_F^2}{\|u_k\|_F^2}, \quad h_k(i) = \mathbf{1}\{\gamma_k(i) > \theta_k\}, \quad (\text{E.3})$$

572 the fraction of injected energy outside high-response regions. Table E.3 reports relative aggregate
 573 diagnostics. The single-role variants affect complementary diagnostics, and full FlowTrack combines
 574 them in one execution rule.

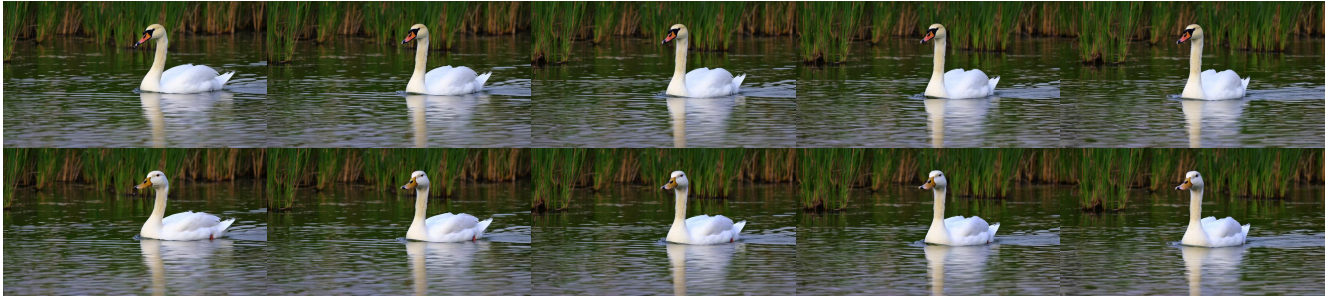
Table E.3: Execution-drift diagnostics. Entries are reported relative to the direct-execution baseline, set to 1.0 for each metric; lower is better.

Method	Endpoint drift↓	Turn variation↓	Response leakage↓
Direct execution	1.000	1.000	1.000
Causal direction only	0.374	0.143	0.905
Support only	0.962	1.041	0.008
FlowTrack (ours)	0.365	0.142	0.009

575 **E.2 Additional Qualitative Examples**

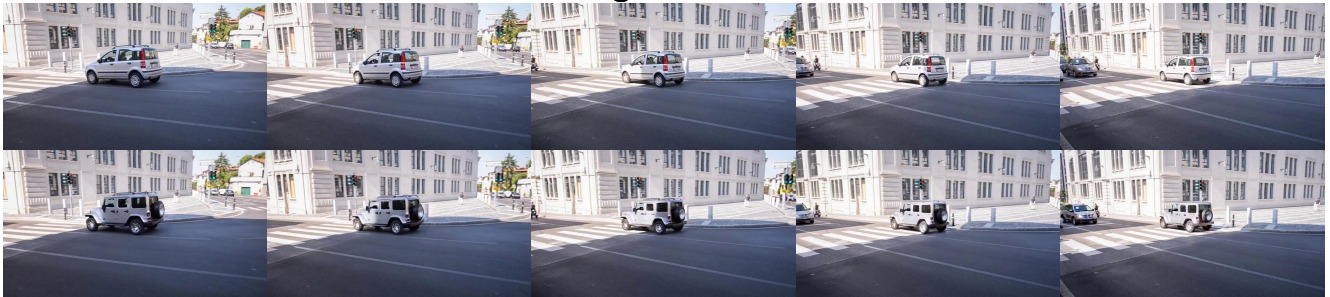
576 The following pages provide additional qualitative examples.

Source: "A **swan** is swimming ..."



Target: "A **duck** is swimming ..."

Source: "A silver **car** is making a turn at an urban intersection ..."



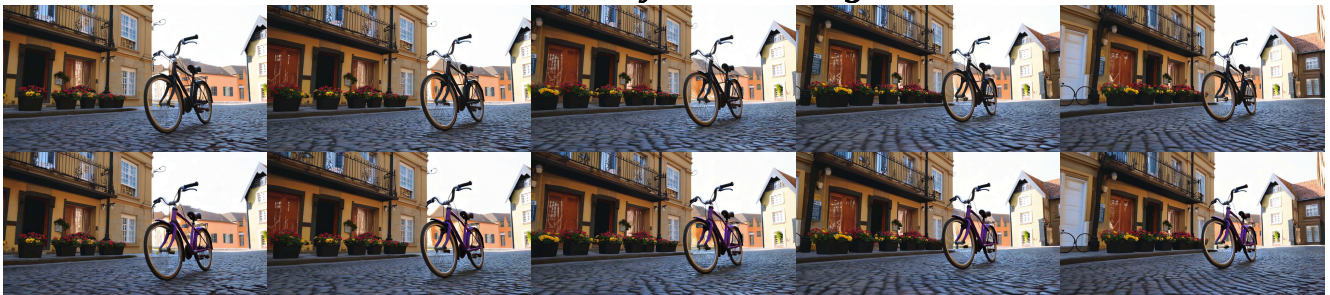
Target: "A silver **jeep** is making a turn at an urban intersection ..."

Source: "A dog is wagging its tail ..."



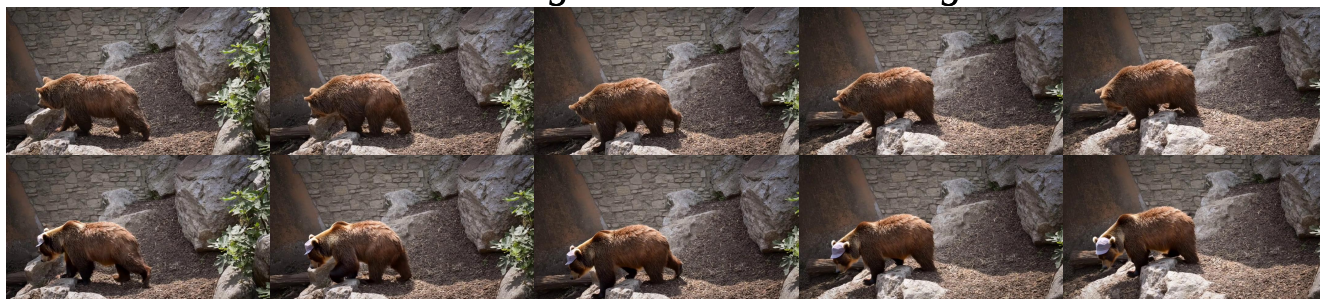
Target: "A dog **wearing sunglasses** is wagging its tail ..."

Source: "A bicycle is rolling ..."



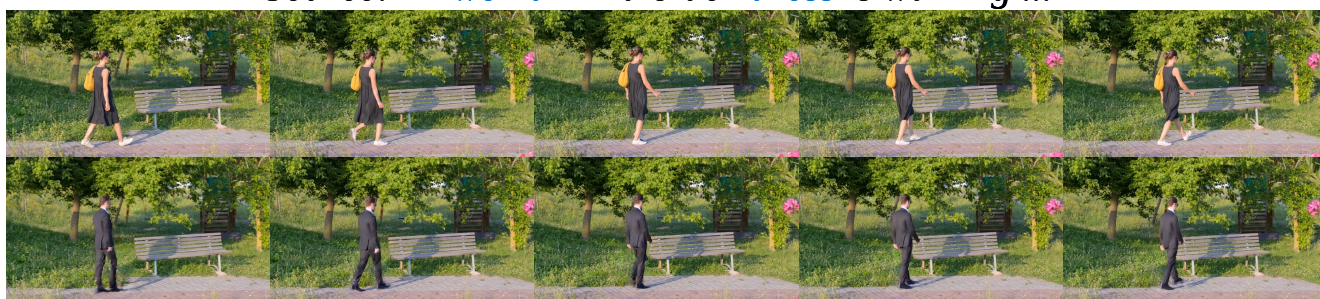
Target: "A **purple** bicycle is rolling ..."

Source: "A large brown bear is walking ..."



Target: "A large brown bear **wearing a cap** is walking ..."

Source: "A **woman** in a black **dress** is walking ..."



Target: "A **man** in a black **suit** is walking ..."

Source: "A **hawk** is soaring ..."



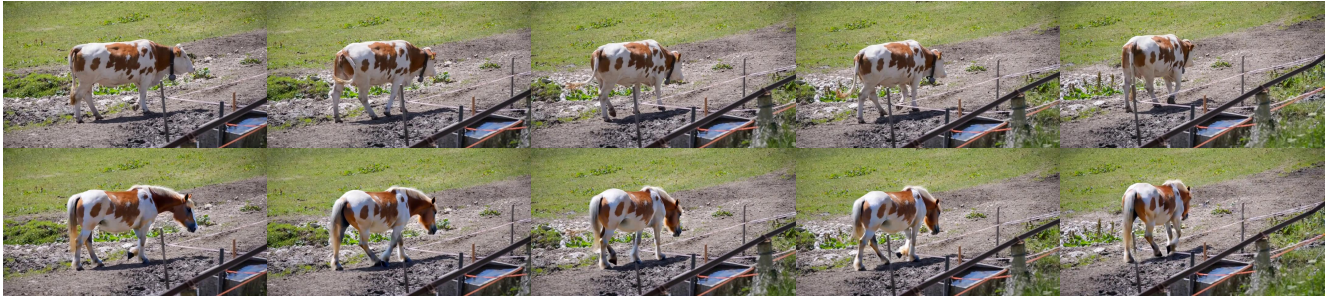
Target: "A **blue bird** is soaring ..."

Source: "A **silver** SUV is navigating ..."



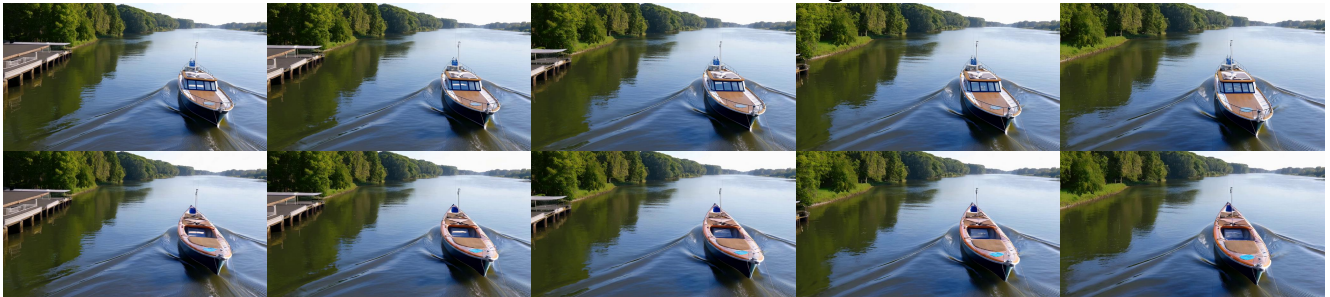
Target: "A **pink** SUV is navigating ..."

Source: "A brown and white **cow** is walking ..."



Target: "A brown and white **horse** is walking ..."

Source: "A **boat** is sailing ..."



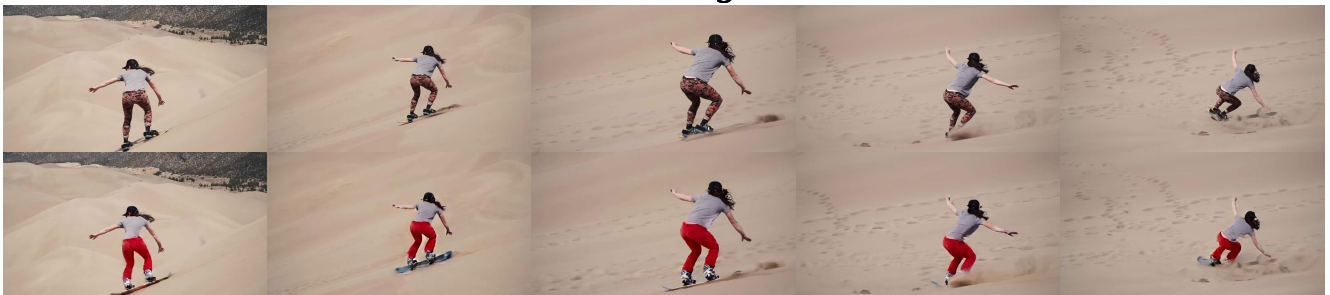
Target: "A **kayak** is sailing ..."

Source: "A **person** on rollerblades is skillfully maneuvering a hockey stick ..."



Target: "A **batman** on rollerblades is maneuvering a hockey stick ..."

Source: "A woman is snowboarding down a vast sand dune ..."



Target: "A woman **in red pants** is snowboarding down a vast sand dune ..."

579

580 E.3 Per-Edit Main Benchmark Breakdown

581 Tables E.4–E.9 report the Edit1–6 breakdown for the same method set and the same metrics as the
 582 overall comparison in Table 1. Best and second-best results are computed within each edit type.

Table E.4: Per-edit FiVE-Bench results for Edit1: rigid object replacement.

Method	MFS↑	CLIP↑	PSNR↑	LPIPS↓	MSE↓	SSIM↑	Structure↓
FateZero	73.4755	<u>28.4258</u>	18.5602	239.0035	157.1404	69.0749	55.2132
FLATTEN	73.4665	29.4946	16.3837	269.4556	254.8604	62.9154	71.9326
FlowDirector	<u>85.3161</u>	27.6062	<u>27.6218</u>	<u>49.9499</u>	<u>21.3606</u>	<u>88.0637</u>	<u>8.7482</u>
RAVE	82.2309	27.3405	20.7481	135.3013	97.9785	78.4373	25.7775
TokenFlow	81.0563	27.3257	22.3862	141.0341	68.6388	75.2023	28.9735
Wan-Edit	83.3070	26.4140	25.4872	71.0405	38.6097	83.5400	12.3476
DFVEdit	82.7405	26.8731	25.1028	72.4702	38.2293	82.8687	12.3172
FlowTrack (ours)	85.9004	27.4159	29.2227	39.5062	15.7261	90.2651	6.4863

Table E.5: Per-edit FiVE-Bench results for Edit2: non-rigid object replacement.

Method	MFS↑	CLIP↑	PSNR↑	LPIPS↓	MSE↓	SSIM↑	Structure↓
FateZero	73.7175	25.6666	18.6434	236.2469	154.3964	69.4072	55.6341
FLATTEN	73.8272	24.6346	16.2663	269.3000	262.2268	62.8539	72.6748
FlowDirector	<u>84.3527</u>	26.9946	<u>26.4601</u>	<u>57.8352</u>	<u>28.6093</u>	<u>86.9424</u>	12.3751
RAVE	82.0287	25.2980	20.8876	140.3730	94.7948	78.1651	28.0525
TokenFlow	81.0647	24.5466	22.7443	137.1591	63.8538	75.5850	26.4307
Wan-Edit	82.9015	23.6471	25.5798	71.9640	40.2110	83.2972	<u>12.3532</u>
DFVEdit	82.4075	25.8303	24.8189	74.7329	40.0733	82.7224	12.5364
FlowTrack (ours)	85.4834	<u>26.4496</u>	28.8211	40.4670	16.6259	90.0871	7.1391

Table E.6: Per-edit FiVE-Bench results for Edit3: color attribute editing.

Method	MFS↑	CLIP↑	PSNR↑	LPIPS↓	MSE↓	SSIM↑	Structure↓
FateZero	73.6010	24.5542	18.6210	237.3585	155.8531	69.3008	55.4188
FLATTEN	74.0191	27.9359	16.3702	270.4217	254.5669	63.0835	70.8870
FlowDirector	<u>85.4753</u>	<u>28.0005</u>	<u>27.7943</u>	<u>51.3024</u>	<u>21.1025</u>	<u>87.9615</u>	<u>7.7401</u>
RAVE	82.1340	28.9285	20.7887	136.1259	96.6058	78.4648	24.6841
TokenFlow	80.7917	26.5397	23.4302	135.0279	50.2260	75.6406	25.3731
Wan-Edit	82.9403	26.9374	25.5959	71.3393	40.7034	83.4429	12.4602
DFVEdit	82.8405	27.9445	25.2454	71.9965	37.2493	83.0204	11.9628
FlowTrack (ours)	86.1796	27.9551	29.5464	39.0453	14.6204	90.2843	5.3443

Table E.7: Per-edit FiVE-Bench results for Edit4: material or texture editing.

Method	MFS $\uparrow$	CLIP $\uparrow$	PSNR $\uparrow$	LPIPS $\downarrow$	MSE $\downarrow$	SSIM $\uparrow$	Structure $\downarrow$
FateZero	73.7651	25.8322	18.6541	235.7857	154.4075	69.3347	55.3841
FLATTEN	74.0599	26.2209	16.3828	272.3117	253.4500	63.0148	71.6602
FlowDirector	<u>85.2317</u>	28.2611	<u>27.5275</u>	<u>51.9785</u>	<u>22.0955</u>	<u>87.8212</u>	<u>7.7177</u>
RAVE	<u>82.3698</u>	28.2196	<u>20.6949</u>	<u>135.6852</u>	<u>100.8704</u>	<u>78.2531</u>	23.6662
TokenFlow	80.7925	27.4095	23.4302	135.0290	50.2264	75.6407	25.3730
Wan-Edit	82.8569	27.5202	25.6140	71.1582	40.4259	83.4266	12.2395
DFVEdit	82.9594	28.0786	25.3257	71.4117	36.7898	83.1159	11.2611
FlowTrack (ours)	86.2942	<u>28.2560</u>	29.6621	38.7390	14.2498	90.3384	5.0324

Table E.8: Per-edit FiVE-Bench results for Edit5: object addition.

Method	MFS $\uparrow$	CLIP $\uparrow$	PSNR $\uparrow$	LPIPS $\downarrow$	MSE $\downarrow$	SSIM $\uparrow$	Structure $\downarrow$
FateZero	81.6077	23.0396	17.9453	236.6002	191.5811	61.5576	56.4715
FLATTEN	86.7598	24.0339	16.7304	268.6913	229.1915	58.1335	61.9898
FlowDirector	95.7084	26.5574	<u>27.7151</u>	<u>52.7001</u>	19.2845	<u>85.9214</u>	<u>5.8687</u>
RAVE	93.9079	25.4578	20.5534	175.1874	109.0435	68.9929	19.7474
TokenFlow	93.1277	24.8591	22.9799	167.4553	53.9430	69.2792	33.3354
Wan-Edit	93.3915	26.0816	24.8548	72.9187	40.1096	80.4392	10.2507
DFVEdit	91.6728	<u>26.8984</u>	24.4012	77.2411	42.6256	79.5563	12.5825
FlowTrack (ours)	<u>95.6919</u>	27.1429	27.9712	47.6488	<u>19.8085</u>	87.5497	5.3290

Table E.9: Per-edit FiVE-Bench results for Edit6: object removal.

Method	MFS $\uparrow$	CLIP $\uparrow$	PSNR $\uparrow$	LPIPS $\downarrow$	MSE $\downarrow$	SSIM $\uparrow$	Structure $\downarrow$
FateZero	70.2418	27.4651	18.9123	228.2837	142.3143	72.7507	53.2384
FLATTEN	65.6185	26.5069	16.8845	255.5012	245.6282	67.8825	63.2281
FlowDirector	<u>81.7510</u>	26.2563	<u>29.5134</u>	<u>41.5544</u>	<u>14.1717</u>	<u>90.2279</u>	5.3580
RAVE	<u>78.3723</u>	26.6825	<u>22.3813</u>	148.1961	<u>62.8719</u>	<u>78.3110</u>	19.5854
TokenFlow	77.5618	<u>27.1500</u>	24.2121	129.1117	42.1802	78.8797	23.4769
Wan-Edit	79.0193	25.9672	26.2420	62.5106	34.4071	85.4920	11.9717
DFVEdit	78.7987	26.3640	26.8107	60.5351	27.4350	86.2045	9.4893
FlowTrack (ours)	82.3038	26.5144	29.8111	36.3013	13.8577	91.2212	<u>5.7764</u>

583 **F Broader Impacts**

584 FlowTrack improves controllability and source-motion/layout stability for text-guided video editing.
585 Potential positive uses include creative video production, education, accessibility, and low-cost
586 content adaptation in cases where keeping the source motion and layout stable is important. The
587 same capability can also be misused to create misleading edited videos, impersonation content, or
588 unauthorized edits of people and copyrighted material. We therefore recommend using FlowTrack
589 with provenance tracking, watermarking or disclosure mechanisms where appropriate, content
590 moderation, and respect for dataset, model, and subject-consent constraints. Any accompanying
591 implementation is intended for research use and evaluation under the licenses and terms of the
592 underlying models and benchmarks.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes].

Justification: The abstract and introduction state the method scope as an execution controller for inversion-free flow video editing, and the experiments in Section 5 evaluate the claimed motion, unedited-region, structure, and prompt-alignment behavior. The conclusion also states the main limitations, including dependence on the pretrained generator and the causal support proxy.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes].

Justification: Section 6 discusses limitations, including dependence on the pretrained generator, reliance on a causal response proxy for support estimation, and the need for richer support fields for multi-object and long-horizon edits.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes].

Justification: The theoretical claims are stated with their assumptions in Section 4 and Appendices A–D. Proofs are provided for the shared-noise identities, conditional-mean interpretation, closed-form optimizer, causal policy property, and support-cost equivalence.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes].

Justification: Section 5 and Appendix E specify the benchmark protocol, metrics, ablation settings, default support estimator, and memory strength used for the main experiments. The method update and support estimator are fully described in Section 4 and Appendix D.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes].

Justification: The supplemental material provides implementation and data-access instructions for reproducing the main evaluations, including benchmark preparation notes. Materials are provided in anonymized form for submission where applicable.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes].

Justification: Section 5 and Appendix E specify the FiVE-Bench Edit1–6 protocol, weighted Overall aggregation, evaluation metrics, matched ablation protocol, Otsu response support field, and default $\lambda = 0.20$. The method is training-free, so there is no optimizer or training split specific to FlowTrack.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes].

Justification: Appendix ?? reports 95% stratified paired-bootstrap confidence intervals over the FiVE-Bench examples, preserving the Edit1–6 composition, together with auxiliary approximate two-sided bootstrap p -values for representative main and ablation comparisons.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes].

Justification: Appendix E reports the GPU class, memory usage, representative shard runtime, training-free nature of FlowTrack, and parallelization over edit types and examples.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes].

Justification: The work uses existing generative video-editing models and an established benchmark for algorithmic evaluation, and does not involve deceptive data collection, human-subject experiments, or release of private data. The submission is anonymized for review.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.

- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes].

Justification: Appendix F discusses positive uses for creative, educational, accessibility, and adaptation workflows, potential misuse for misleading or unauthorized video edits, and mitigation practices such as provenance, watermarking, disclosure, moderation, and respecting licenses and consent.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A].

Justification: The paper does not introduce or release a new pretrained video generator, high-risk dataset, or scraped asset collection. The method is an execution controller evaluated with existing models and benchmarks.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes].

Justification: The paper cites the existing benchmark, pretrained model family, and compared methods used in the experiments. The accompanying materials document the relevant licenses and terms of use for benchmark data, pretrained models, evaluation assets, and implementation resources.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes].

Justification: The paper introduces an implementation of the FlowTrack execution controller and documents it with reproduction instructions. No new dataset or pretrained model is introduced.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A].

Justification: The paper does not involve crowdsourcing, user studies, or other human-subject experiments. Evaluation is performed with automated benchmark metrics.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.

- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A].

Justification: The work does not involve human-subject research or crowdsourcing, so IRB approval or equivalent review is not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A].

Justification: The core method does not use an LLM as a methodological component. Any use of language tools for writing, editing, or formatting would not affect the method, experiments, or scientific claims.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.